Image Preprocessing in the G9 Hybrid RAG Project

Oct 30, 2026 · @hmidadda

Table of contents

- Executive summary
- 1. Introduction
- 2. Key concepts in plain language
- 3. The dataset and the three splits
- 4. The preprocessing pipeline at a glance
- 5. Data audit and integrity checks
- 6. Preventing data leakage
- 7. Image preprocessing
- 8. Caption preprocessing
- 9. Exploring the data: objects and scenes
- 10. Exploring the data: language
- 11. Diversity and relationships
- 12. Metadata extraction
- 13. Turning images and words into vectors
- 14. Building the knowledge base
- 15. Testing the whole chain
- 16. Quality control and audit results
- 17. Limitations and open issues
- 18. Conclusion and next steps
- Appendix A: Technical reference

Executive summary

Group G9 built the data foundation of a hybrid retrieval-augmented image captioning system: 10,200 COCO pictures and their sentences were selected, checked, cleaned, turned into numbers and stored in a searchable library, and all 13 automated audit checks pass.

In plain terms, a computer that describes a photograph by looking up similar photographs cannot start from raw files. Someone must decide which pictures belong in its library, make sure none of the exam pictures are hiding in it, repair or reject damaged files, clean the text, and translate everything into a form a machine can compare quickly. That preparation is called preprocessing, and it is the subject of this report.

What G9 delivered

- **Three separate sets of pictures.** A knowledge base of 10,000 pictures with 50,000 descriptions (five per picture), a validation set of 100 pictures and a test set of 100 pictures. They share no picture, and the random draw can be repeated exactly because its seed (42) is recorded.
- **A clean and verified image collection.** All 10,200 pictures were opened, turned the right way up, converted to standard colour and resized for the CLIP model. None failed.
- **Two versions of every sentence.** The original text is kept untouched for evaluation. A cleaned copy is used only for searching.
- **A statistical portrait of the data.** The knowledge base covers all 80 COCO object categories, averages 7.21 annotated objects per picture, and its captions average 10.49 words with a vocabulary of 9,770 words.
- **A searchable library.** Each picture and each sentence is represented by 512 numbers. Three indexes (two for meaning, one for exact words) were built, each in about one second or less.
- **An audit.** The automated audit checks counts, overlaps, file integrity, vector quality and duplicate images. All 13 checks pass.

Status. The release is ready for review but deliberately not frozen. Two points need the teacher′s decision before the freeze: the assignment gives two different sizes for the test set (100 and 1,000 pictures) and asks for a stratified test set, while G9′s current test set is a random draw of 100. A third step, using the 100 validation pictures to compare preprocessing choices, remains to be done.

How to read this report. Chapters 1 to 4 explain the ideas and the overall plan without any programming. Chapters 5 to 17 follow the pipeline stage by stage. Most chapters end with a short paragraph headed Technical note for readers who want the exact tools and settings; the full reference is in Appendix A.

1. Introduction

This mini-project asks every group to build a system that writes a sentence describing a photograph, and G9′s share of that work is the preparation of the data on which the system stands.

1.1 The project in one paragraph

The course project is organised around a technique called retrieval-augmented generation, usually shortened to RAG. Instead of asking a model to describe a picture from nothing, the system first looks for pictures that resemble it in a large library, reads the descriptions that people wrote for those similar pictures, and uses that evidence to help produce a better description. The assignment defines ten RAG techniques, and each technique is given to three groups so that their results can be compared fairly. Groups G7, G8 and G9 share the technique called Hybrid RAG, which combines two ways of searching: one based on the meaning of pictures and sentences, and one based on the exact words they contain.

All groups use the same dataset. The assignment states that the dataset is the best-performing output of a common preprocessing stage, so that differences between techniques come from the techniques themselves and not from differences in the data. This is why preprocessing is not a minor chore: every group's results depend on it.

1.2 Objectives

The assignment (section 4.4) describes the goal as transforming raw COCO data into a clean, reproducible and shared set of artifacts. G9 translated this into eight working objectives:

1. Select and fix three non-overlapping sets of pictures: a knowledge base, a validation set and a test set.
2. Audit the data: counts, damaged files, duplicates, and the match between pictures and sentences.
3. Protect the evaluation by preventing leakage.
4. Prepare the pictures so that the image model can read them.
5. Prepare the sentences, keeping the originals untouched.
6. Explore the data statistically and visually (assignment section 4.3).
7. Convert pictures and sentences to numerical vectors and store them in searchable indexes.
8. Control quality with automated checks and an audit record.

1.3 Scope

This report covers the preprocessing phase only. It does not cover training, caption generation or the final evaluation, which belong to later phases of the project; the assignment's timetable gives the data preparation phase a 4 November deadline, after

which the professors validate it, and a 2 December deadline for model training and validation. Where a result is incomplete or a decision is pending, the report says so openly (Chapter 17).

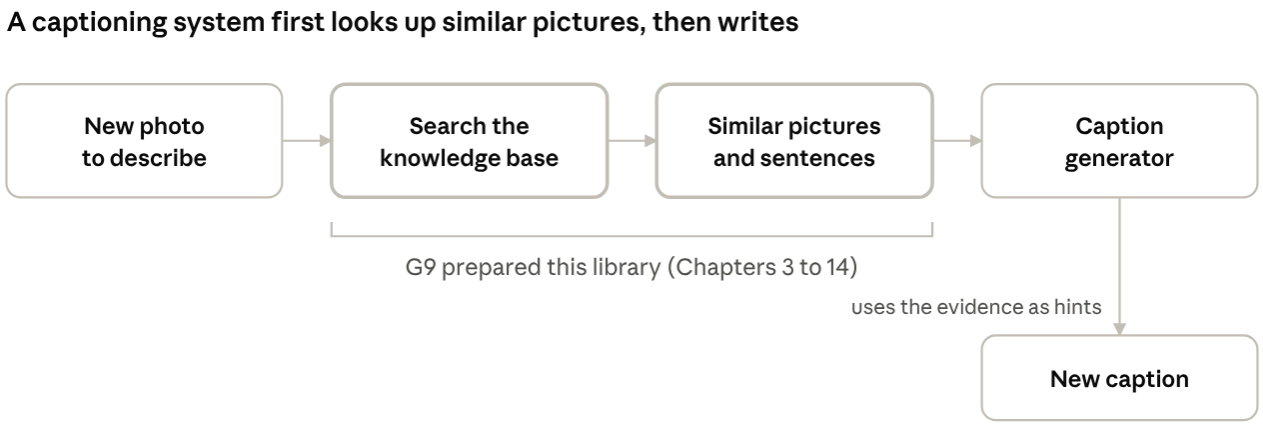
Technical note. The work was carried out in three Python notebooks run on Google Colab (with a Tesla T4 graphics processor for the embedding step): 00_G9_COCO_Dataset_Setup, 01_G9_Image_Preprocessing and G9_Final_Preprocessing_Audit. All data and results are stored in a shared Google Drive folder named G9_Hybrid_RAG, organised into data (splits, annotations, archives, embeddings, indexes, metadata) and results (exploration, quality control, retrieval, release). The main libraries are Pillow (images), Hugging Face transformers (CLIP), FAISS and rank-bm25 (indexes), spaCy (grammar analysis), pandas and matplotlib (statistics and charts).

2. Key concepts in plain language

This chapter explains, without programming, the eight ideas on which the rest of the report rests. A reader who understands these pages can follow every later chapter.

2.1 Describing a picture by looking up similar ones

A captioning model that works alone must remember everything it knows about the world inside its own parameters. A retrieval-augmented system works more like a student who is allowed to consult a library. Given a new photograph, it first searches a library of pictures that already have human-written descriptions, picks the closest matches, and reads their descriptions as hints. Only then does it write its own sentence. Figure 1 shows the idea.



concept diagram, 4 steps

Figure 1. Retrieval-augmented captioning: the system searches a library before it writes.

The library is called the knowledge base. Its quality decides how useful the hints can be, which is why G9′s work concentrates on building it carefully.

2.2 Pictures and words as numbers

A computer does not see a dog or read the word dog. It sees numbers. An embedding is a list of numbers, here 512 of them, that summarises the content of a picture or a sentence. The important property is that similar content gives similar numbers. A useful comparison is a map: on a street map every place has two coordinates, and places that are close on the map are close in real life. An embedding gives every picture and every sentence a position on a much larger map with 512 directions instead of two, and pictures that show similar scenes land near one another. To find similar pictures, the computer simply looks for the nearest points.

2.3 CLIP: one language for pictures and words

G9 uses a model called CLIP (Contrastive Language-Image Pre-training), published by OpenAI. It has two halves, an image encoder and a text encoder, trained together on a very large collection of pictures with their captions so that a photograph and a sentence describing it end up close together on the same map. This is what makes it possible to search sentences using a picture, or pictures using a sentence. The specific version used is ViT-B/32, a medium-sized model that is widely used and runs comfortably on a free graphics processor.

2.4 Two ways of searching

There are two complementary ways to look for relevant sentences.

- **Searching by meaning (dense retrieval).** The query is turned into an embedding and the nearest embeddings are returned. It understands that a puppy resting near a bike and a dog sleeping by a bicycle are similar even though few words are shared. It can, however, miss a rare but important word.
- **Searching by words (sparse retrieval).** The classic method is BM25, the same family of ideas as the index at the back of a book. It counts how often the query words appear in each sentence and gives more weight to words that are rare in the collection. It is precise on exact words but blind to synonyms.

Each method has a weakness that the other covers.

2.5 Combining them: hybrid retrieval

Hybrid RAG, G9′s assigned technique, runs both searches and merges the two ranked lists. The merging method used in G9′s test is reciprocal rank fusion. Think of several judges who each rank the candidates; every judge gives a candidate points that shrink as

its rank gets lower, and the points are added up. A picture ranked highly by several judges wins, even if no single judge put it first. Its appeal is that it needs no tuning of scores between very different methods.

Technical note. Similarity between embeddings is measured by cosine similarity; because every vector is scaled to length one (L2 normalisation), cosine similarity equals the inner product, which is what the FAISS index computes. BM25 is the Okapi variant implemented in the rank-bm25 package. Reciprocal rank fusion gives each candidate the score sum over rankings of 1 / (k + rank), with k = 60 in G9′s test. CLIP ViT-B/32 refers to a vision transformer that cuts the picture into 32 by 32 pixel patches and produces 512-dimensional vectors for both pictures and text.

3. The dataset and the three splits

G9 drew 10,200 pictures at random from the COCO 2017 collection and divided them into a knowledge base, a validation set and a test set that share nothing.

3.1 What COCO contains

COCO (Common Objects in Context) is a public collection of photographs of everyday scenes, created for research on image understanding. Each photograph comes with two kinds of human-made information that matter here.

- **Captions.** Several people independently wrote a one-sentence description of the picture. Most pictures have five. These sentences are the raw material that the retrieval system will search.
- **Object annotations.** For every visible object of 80 common categories (person, car, chair, dog, cup and so on), annotators marked its position with a bounding box and an outline. They tell us what is in each picture and how many objects there are.

The 2017 edition is divided by its authors into a large training part of 118,287 pictures and a validation part of 5,000 pictures. G9′s first notebook checks these two counts exactly, and also checks that the identity numbers of the pictures are unique and that the two parts do not overlap.

3.2 Choosing the sizes

The assignment prescribes the sizes: 10,000 pictures for the knowledge base and 100 for validation. A knowledge base of 10,000 pictures with five sentences each (50,000 sentences) is small enough to process in minutes on a free cloud computer and large enough to contain varied scenes. For the test set the assignment mentions both 100 and 1,000 pictures in different places; G9 used 100 for now and flags the contradiction as an open decision (Chapter 17).

3.3 How the random draw was made

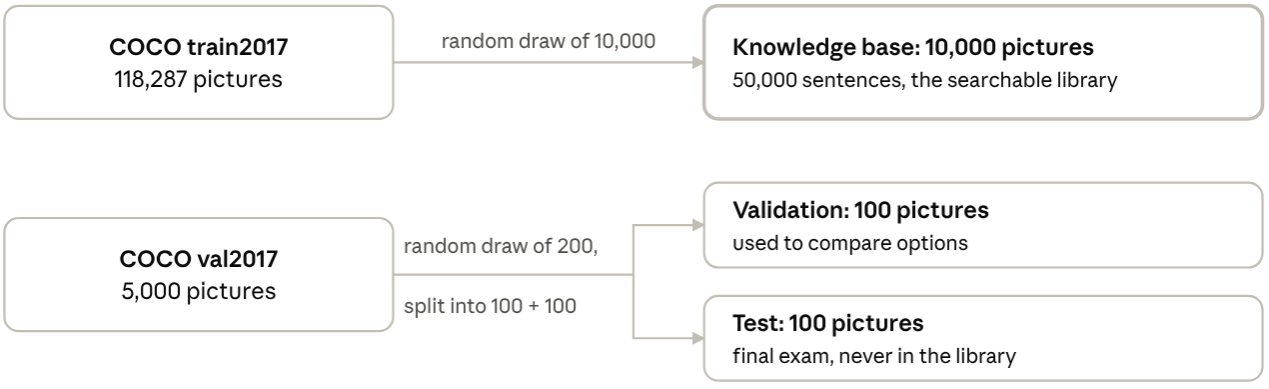
The selection is random, but it is repeatable. A random number generator can be started from a seed, a fixed starting number, after which it always produces the same sequence of choices. Think of rolling a die that has been rigged to repeat the same rolls every time the experiment is run: it looks random but is reproducible, and anyone who uses seed 42 obtains the same 10,200 pictures. G9 recorded this seed in the split file itself.

The steps were as follows.

1. The identity numbers of all training pictures were sorted, and 10,000 were drawn for the knowledge base.
2. From the 5,000 pictures of the COCO validation part, one picture (number 139) was set aside because it had been used earlier as a learning example. From the rest, 200 distinct pictures were drawn.
3. The 200 pictures were divided in two: the first 100 became the validation set and the last 100 became the test set.
4. Several automatic checks confirmed that the sizes are right, that no picture appears twice, and that the three sets do not overlap. Altogether there are exactly 10,200 distinct pictures.

Figure 2 summarises the structure. A key design choice is that the knowledge base comes from one COCO part and the validation and test pictures from the other. Because the two parts are separate by construction, a test picture can never be in the knowledge base, which is the strongest possible protection against leakage.

The library and the exam pictures come from different COCO parts



No picture appears in two sets, so a test picture can never be in the library.

diagram, 2 sources and 3 sets

Figure 2. How COCO was divided into the knowledge base, the validation set and the test set.

3.4 What was recorded

The result is stored in small, versioned files rather than only in someone's memory.

Split	Source part	Pictures	Descriptions
Knowledge base	train2017 (118,287 pictures)	10,000	50,021 original, 50,000 used for retrieval
Validation	val2017 (5,000 pictures)	100	501
Test	val2017	100	kept apart, not part of the approved annotation files

A split file lists the identity number of every picture in every set, together with the seed. A manifest file has one row per picture (10,200 rows) giving its set, its identity number, its file name and its source part. The pictures themselves are stored in uncompressed archive files, and the descriptions and object annotations are filtered so that each approved file contains only the pictures of its own set.

Technical note. The draw uses Python's random.Random(42) with rng.sample on sorted identifier lists; the knowledge base is sorted after sampling. The files are data/splits/split_ids.json and data/splits/image_manifest.csv (columns split, image_id, file_name, source). Pictures were downloaded from the official COCO server with retries and a JPEG check, grouped in archives of 250 pictures (40 for the knowledge base, 1 for validation, 1 for test) and stored uncompressed (ZIP_STORED). Each archive is verified for file names and CRC integrity. The approved annotation files hold 50,021 captions and 72,062 object annotations for the knowledge base and 501 captions and 778 object annotations for validation.

4. The preprocessing pipeline at a glance

G9's preprocessing is a chain of twelve stages, each of which produces a file that the next stage uses and an automatic check that proves the stage worked.

4.1 The big picture

Figure 3 shows the whole journey from the raw COCO collection to the shared release. The chain can be read as a factory line. Raw material enters on the left: the official COCO annotation files and photographs. Along the line the material is selected, inspected, cleaned, measured and converted into the form that the retrieval system needs. At the end, a quality inspection decides whether the product can be released to the other groups.

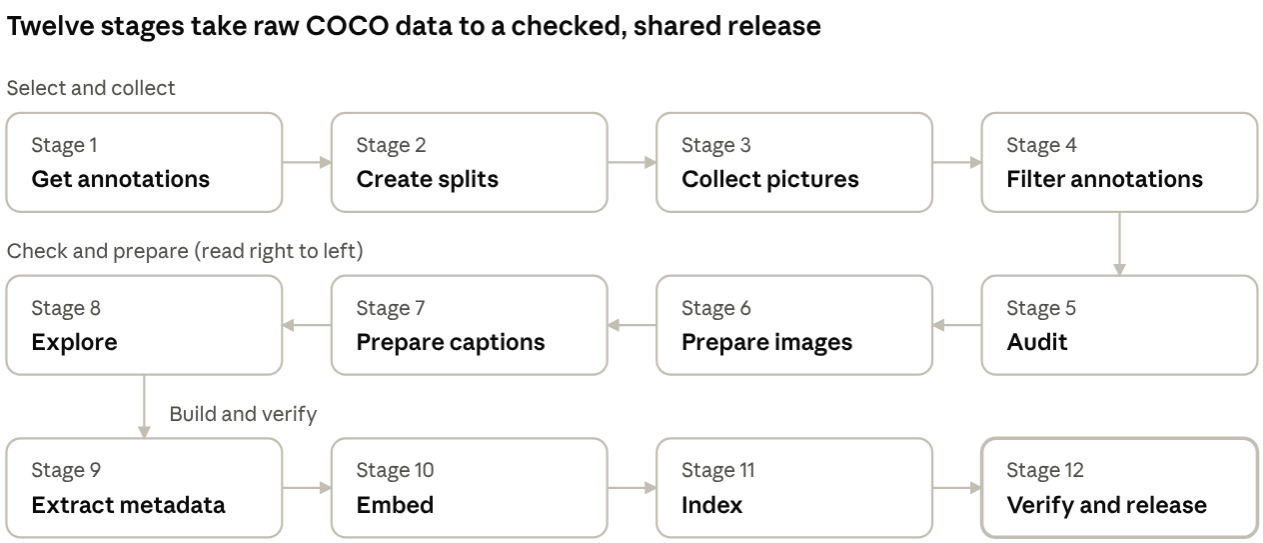


Figure 3. The twelve stages of the preprocessing pipeline.

4.2 Four working principles

Several principles guided every stage, and they explain why the pipeline can be trusted.

- **Originals are never modified.** The original captions and the original annotation files are kept as they are. Cleaned versions are stored beside them, in separate columns or files, with clear names.
- **Every stage is checked by the computer.** Counts, overlaps, shapes and file integrity are verified by programs, not by eye. A stage that fails stops with an error message instead of silently continuing.
- **Work can be resumed.** Heavy steps such as downloading pictures or computing vectors save their results in pieces. If the cloud session is interrupted, the next run skips what is complete and continues.
- **Everything important is written down.** Settings such as the seed, the model name and the vector size are stored in files next to the results, so someone else can see exactly how they were produced.

Technical note. The Google Drive project folder contains two main branches. Under data: splits, annotations (original COCO files), approved_annotations, images_archives, preprocessed_captions, metadata, embeddings/clip_vit_base_patch32 and indexes. Under results: eda (statistics and charts), qa (quality reports), retrieval (smoke-test outputs) and preprocessing_release (final audit and its README). Stages 1 to 4 are implemented in notebook 00, stages 5 to 11 in notebook 01, and stage 12 in the final audit notebook.

5. Data audit and integrity checks

Before any picture was processed, G9 audited the collection and found no missing, damaged or duplicated picture among the 10,200 selected.

5.1 What was verified

- **Counts.** The official collection must contain 118,287 training pictures and 5,000 validation pictures, and the selection must contain exactly 10,000, 100 and 100.
- **Identity.** Every picture number must be unique, and no number may appear in two sets.
- **Completeness of the files.** Every file listed in the manifest must exist in the archives, no unexpected file may be present, and no file may appear twice. Each archive must also pass an internal integrity test.
- **Readability.** Every picture must open as a valid JPEG and be decoded completely.
- **Sentences.** Every knowledge-base picture must have at least five sentences, and every sentence must belong to a knowledge-base picture.
- **Duplicates.** No two pictures from different sets may be byte-for-byte identical.

5.2 The 102 pictures without object annotations

For 102 knowledge-base pictures, COCO contains no labelled object. This is a property of the original collection and not an error: for example, a scene may contain nothing from the 80 categories. These pictures remain in the knowledge base with their five sentences, and the statistics report them as a separate group (Chapter 9) so that averages are not distorted.

Technical note. Archive checks compare the names stored in each ZIP file with the expected names derived from the manifest (split/file_name) and run ZipFile.testzip() for CRC verification. The cross-set duplicate search first groups all files by their stored CRC-32 value and size, and computes SHA-256 only for groups that span more than one set; the list of duplicate groups is empty. Picture decoding uses Pillow with a JPEG format assertion. The record of the image audit is results/qa/image_preprocessing_audit.csv, with one row per picture and the columns split, image_id, file_name, archive, original_width, original_height, original_mode, exif_orientation, processed_shape, status and error; status can be PASS, FAIL or MISSING.

6. Preventing data leakage

G9 protects the final evaluation with four layers, so that no test picture and no test sentence can reach the knowledge base.

6.1 What the assignment requires

The assignment sets three rules. First, the reference sentences of test pictures must never be included in the knowledge base. Second, a test picture must never be retrieved for itself. Third, the identity numbers of each set must be stored in versioned files and the leakage audit must be documented. The last paragraph of section 4.4.4 adds that no group may modify the frozen test set or use test reference sentences during retrieval or generation.

The reason for this strictness is the exam analogy of Chapter 2. If the answer sheet is in the library, the measured score says nothing about the quality of the system.

6.2 Four layers of protection

Layer 1: separate sources. The knowledge base comes from the COCO training part and the validation and test pictures come from the COCO validation part. Two parts that were separated by the creators of the dataset cannot contain the same picture. This makes the second rule true by construction: a test picture is not in the library, so it cannot be returned as its own neighbour.

Layer 2: filtered annotation files. After the draw, each approved annotation file was cut down to the pictures of its own set, and the program verified that no sentence or object belongs to a picture outside that set. Only knowledge-base and validation annotation files exist in the approved folder. The test sentences are not part of any file used by the retrieval pipeline.

Layer 3: frozen identifiers. The list of pictures of each set is written once to the split file. If the notebook is run again, it does not silently overwrite the file: it compares the new selection with the saved one and stops with a warning if they differ. This prevents someone from accidentally changing the test set after other work depends on it.

Layer 4: automated audit. The final audit repeats the checks (counts, no overlap, sentences scoped to their sets) and adds a search for byte-identical pictures across sets. The result is recorded in a file that other groups and the teachers can read.

In addition, the retrieval test described in Chapter 15 used a test picture only as a query picture. Its reference sentences were not read, and the configuration file of that run records this explicitly (uses_test_reference_captions is false).

Technical note. The scoping check asserts that every annotation image_id belongs to the set of its own file and that the images listed in each approved file equal the expected identifiers. The overwrite guard loads the existing split_ids.json and asserts equality with the in-memory selection before writing. Exact duplicate detection groups ZIP entries by the pair (CRC-32, file size) and confirms candidates with SHA-256; the audit stores the result under exact_cross_split_duplicate_groups. The split audit record also states

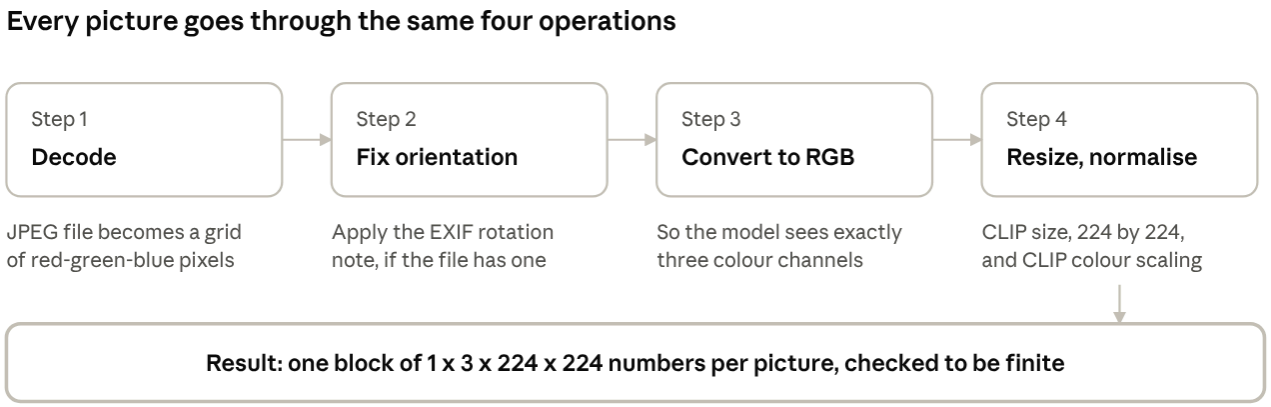
split_strategy as random seed 42, stratification as not verified, and content_duplicate_check as not performed at that stage; the later audit notebook performs the exact-duplicate check.

7. Image preprocessing

Every one of the 10,200 pictures passes through the same four operations, which turn a photograph in any shape into the standard input that the CLIP image model expects.

7.1 The four operations

Figure 4 shows the sequence. Each operation is explained below.



flow diagram, 4 operations

Figure 4. The four operations applied to every picture.

Operation 1: open and decode. A JPEG file is a compressed description of a picture. Decoding it produces a grid of pixels, and each pixel is described by three numbers (the amounts of red, green and blue), each between 0 and 255. The program opens each file from its archive and decodes it completely, which also proves that the file is not damaged.

Operation 2: correct the orientation. Cameras and phones often store a picture in the orientation of the sensor and add a small note, called the EXIF orientation tag, saying how the picture should be rotated when displayed. A program that ignores the note shows or analyses the picture sideways. The assignment explicitly asks for this correction. G9′s program reads the note and rotates the picture accordingly. For the first picture inspected, the tag was absent, meaning no rotation was needed; the step is a safeguard for the pictures that do carry a tag.

Operation 3: convert to RGB. Some images are stored in other colour formats, for example greyscale or with a transparency layer. The model expects exactly three colour channels, so every picture is converted to the standard red-green-blue format. For a picture that is already RGB nothing changes, as in the example below.

Operation 4: resize and normalise for CLIP. CLIP works on pictures of 224 by 224 pixels. Using the official preprocessing tool that comes with the model, each picture is scaled to this size. In the tool's default settings the shorter side is scaled to 224 pixels, which keeps the proportions of the picture, and the central 224 by 224 area is kept. This means that the borders of very wide or very tall pictures can be cut off, a limitation discussed in Chapter 17. The colour values are then normalised: for each channel the average value expected by the model is subtracted and the result is divided by a typical spread. After normalisation, the numbers lie roughly between minus 2 and plus 2 instead of 0 to 255. This is the scale on which the model was trained, so using it gives meaningful results.

7.2 A worked example

The first knowledge-base picture inspected had a size of 500 by 374 pixels, was already in RGB and had no orientation tag. After processing it became a block of numbers with the shape 1 by 3 by 224 by 224 (one picture, three colour channels, 224 rows, 224 columns). The smallest value in the block was about minus 1.78 and the largest about 2.15, which is within the expected range. A visual comparison of the original picture with the model's view of it, obtained by reversing the normalisation, was also made to confirm that the picture is still recognisable.

7.3 Checking all 10,200 pictures

After trying the procedure on single pictures, G9 applied it to every picture of the three sets. For each picture the program recorded its original size, colour mode, orientation tag and the shape of the result, and it verified two conditions: the output has the right shape, and all its numbers are finite (no missing or infinite values). The outcome was 10,200 passes, no failure and no missing picture. The table of results is saved as a file so that anybody can examine any single picture.

Technical note. The function preprocess_image(image_bytes) opens the bytes with Pillow, loads the image, reads the EXIF tag 274 (orientation), applies ImageOps.exif_transpose, converts to RGB, then calls the CLIPImageProcessor of openai/clip-vit-base-patch32, which returns a float tensor of shape (1, 3, 224, 224). It asserts the shape and torch.isfinite for every picture. The same sequence (exif_transpose, convert to RGB, CLIPImageProcessor) is applied again in batches of 32 during embedding generation (Chapter 13), so the audited preprocessing is the preprocessing that produced the vectors. The processor's mean and standard deviation are read from its own configuration (processor.image_mean and processor.image_std).

8. Caption preprocessing

G9 keeps every human-written sentence exactly as it was written and creates a separate cleaned copy that is used only for word-based searching.

8.1 What the sentences look like

Each picture has five sentences written independently by different people. They describe the same scene with different words, which is precisely what makes them useful as evidence. For the first knowledge-base picture inspected (number 74), the five sentences are:

- A picture of a dog laying on the ground.
- Dog snoozing by a bike on the edge of a cobblestone street
- The white dog lays next to the bicycle on the sidewalk.
- a white dog is sleeping on a street and a bicycle
- A puppy rests on the street next to a bicycle.

The example shows the irregularities that cleaning must handle: some sentences start with a capital letter and end with a full stop, some do not, and the same object is called dog, puppy, bike or bicycle depending on the author.

8.2 The principle: raw and cleaned side by side

The assignment is clear: the original reference sentences must stay unchanged for evaluation, and cleaning may be applied for retrieval and indexing only, with both versions stored under clear field names to avoid leakage. The reason is that the final evaluation compares the system's sentence with the original human sentences. If the originals had been altered, the comparison would be unfair or impossible. G9 therefore stores, for every sentence, the original text (caption_raw) and a cleaned version (caption_clean) in separate columns of the same table, together with the picture number and the annotation number.

8.3 The cleaning operations

The cleaning is deliberately light. Three operations are applied in order.

1. **Lowercasing.** Capital letters are turned into small letters, so that Dog and dog count as the same word.
2. **Punctuation removal.** Commas, full stops and other symbols are replaced by a space.
3. **Whitespace normalisation.** Repeated spaces are merged into one and spaces at the ends are removed.

Two examples taken from the knowledge base show the effect.

Original sentence	Cleaned sentence
A bathroom with a toilet, sink, and shower.	a bathroom with a toilet sink and shower
A brown and black horse in the middle of the city eating grass.	a brown and black horse in the middle of the city eating grass

The program also checks that no cleaned sentence became empty.

8.4 What was deliberately not done

Some common text-processing steps were not applied to the indexed text. Stop words (very frequent words such as the or of) were not removed, words were not reduced to their roots (stemming), and spelling was not corrected. The CLIP model benefits from complete natural sentences, and BM25 already gives little weight to very frequent words. Stop-word removal and word roots are used only in the statistical analysis of Chapter 10, never in the library itself.

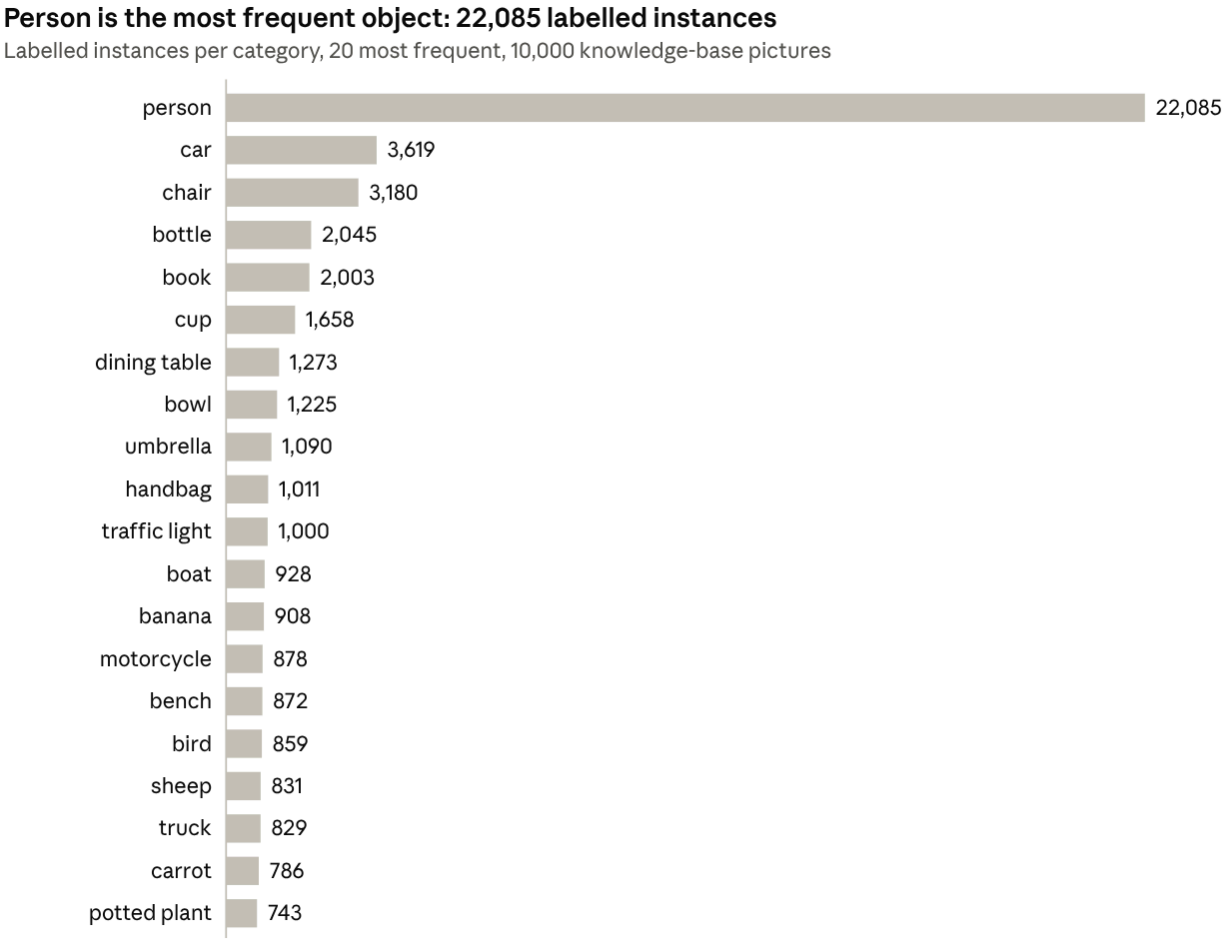
Technical note. The cleaning function lowercases the text, replaces every character that is neither a letter, digit, underscore nor whitespace by a space (a regular expression substitution), and then collapses repeated whitespace and strips the ends. The files are data/preprocessed_captions/kb_captions_processed.csv, kb_captions_retrieval.csv and kb_extra_captions_audit.csv, each with the fields image_id, annotation_id, caption_raw and caption_clean. For each picture, the five sentences with the lowest annotation numbers are retained. The CLIP tokenizer is called with padding, truncation and a maximum length of 77 tokens, so a sentence longer than that would be truncated; the longest cleaned sentence has 46 words. BM25 tokenises the cleaned sentence by splitting on spaces, and queries are tokenised by extracting word characters after lowercasing.

9. Exploring the data: objects and scenes

The 10,000 knowledge-base pictures contain 72,062 labelled objects from all 80 COCO categories, but the collection is dominated by people and varies widely in how crowded its scenes are.

9.1 Which objects are most frequent

Figure 5 shows the twenty most frequent categories. Three observations stand out.



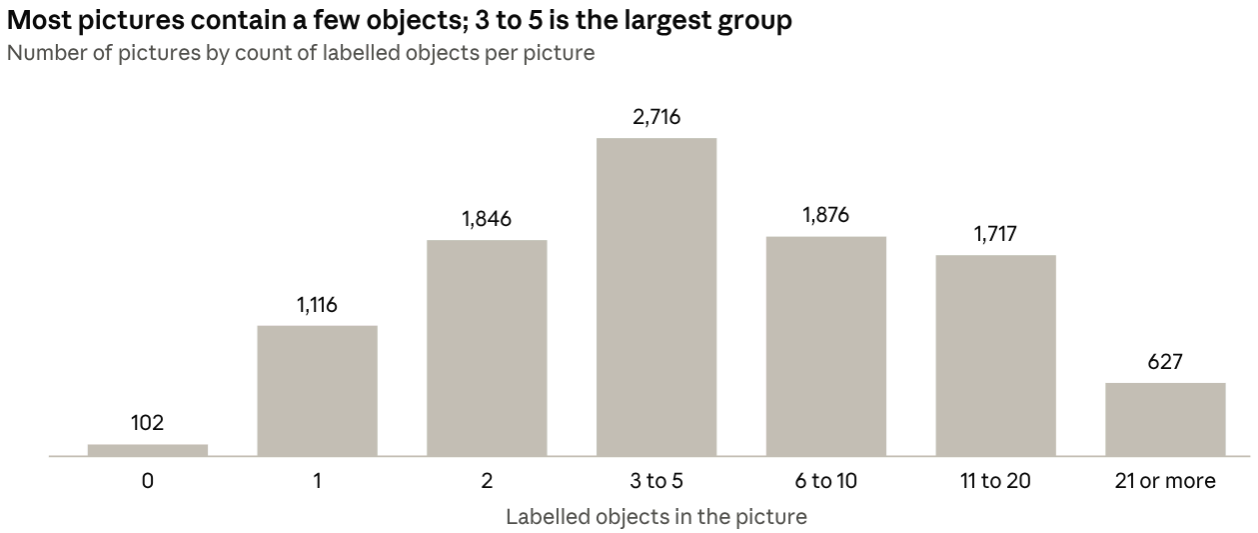
top_50_object_categories.csv, first 20 rows

Figure 5. The twenty most frequent object categories.

- **People dominate.** The category person has 22,085 labelled instances, about 31% of all 72,062, and appears in 5,421 pictures, more than half of the collection. The ten most frequent categories together account for about 54% of all labelled objects.
- **Number of instances is not number of pictures.** A category can be frequent because it appears in many pictures or because it appears many times in a few. Books have 2,003 instances in only 447 pictures, about 4.5 per picture, and sheep have 831 instances in 132 pictures, about 6.3 per picture, because sheep come in flocks. Cars have 3,619 instances in 1,000 pictures.
- **There is a long tail.** After the leading categories, the counts fall gradually: the fiftieth category in the ranking (zebra) still has 453 instances, but many others are much rarer. The retrieval system will therefore find many neighbours for a picture of people in a street and few for a picture of a rare object.

9.2 How crowded are the scenes

The assignment asks for a measure of scene complexity. G9 used the simplest honest measure: the number of labelled objects in a picture. On average a picture has 7.21 labelled objects and 2.87 different categories, but the median is only 4. The average is higher than the median because a minority of very crowded pictures pull it upward: a quarter of the pictures have two objects or fewer, a quarter have ten or more, one picture in ten has at least 17 and the most crowded picture has 56. Figure 6 shows the distribution.



object_count_distribution.csv, 10,000 pictures

Figure 6. Number of labelled objects per picture.

For easier reading, G9 grouped the pictures into four classes. These thresholds are a convenient heuristic chosen by G9, not a COCO definition.

Class	Labelled objects	Pictures	Share
No annotation	0	102	1.0%
Simple	1 to 3	4,118	41.2%
Moderate	4 to 10	3,436	34.4%
Complex	11 or more	2,344	23.4%

Technical note. Object statistics are computed from the file approved_annotations/kb_instances.json (instances_train2017 filtered to the knowledge-base pictures), after asserting that its picture identifiers equal the knowledge-base split and that every category identifier is known. The per-picture table metadata/kb_object_metadata.csv has the columns image_id, object_count, distinct_categories and object_categories. Results are saved in results/eda:

object_category_distribution.csv, top_50_object_categories.csv, object_statistics.json, object_count_distribution.csv and scene_complexity_stats.json (with percentiles 25, 50, 75, 90, 95 and 99 equal to 2, 4, 10, 17, 22 and 33 objects). The object-count bins used in Figure 6 are 0, 1, 2, 3 to 5, 6 to 10, 11 to 20 and 21 or more.

10. Exploring the data: language

The 50,000 knowledge-base sentences are short (about 10 words), built from a small repetitive vocabulary of 9,770 words, and dominated by nouns that name people, places and objects.

10.1 How long are the sentences

The average sentence has 10.49 words and the median is 10, which means that half of the sentences have ten words or fewer. The shortest has 6 words and the longest 46, and the standard deviation of 2.41 words shows that most sentences are close to the average. In total the library contains 524,391 words. This tells us that COCO sentences are concise, single-statement descriptions, not stories. A caption generated by the final system should be of similar length to be comparable with the references.

10.2 How rich is the vocabulary

The 524,391 words are made of only 9,770 different words. Put differently, each distinct word is used on average more than fifty times. About 39% of the distinct words (3,850) appear exactly once, typically rare names, unusual objects or spelling variations. Removing the most common function words such as the or of, for analysis only, leaves a content vocabulary of 9,529 words.

The most frequent content words describe people and what they do: man (6,261 times), sitting (4,491), standing (3,729), people (3,567), white (3,070), woman (3,045), street (2,533), table (2,528), holding (2,375) and person (2,121). In other words, a typical COCO sentence says who is in the picture, what the person is doing, and where.

10.3 What kinds of words are used

To study grammar, G9 used a language tool (spaCy) that assigns every word its part of speech, such as noun or verb. Figure 7 shows the distribution. Nouns make up 34.5% of all words, followed by determiners such as a and the (20.5%), prepositions such as on and next to (17.0%), verbs (9.4%) and adjectives (7.1%).

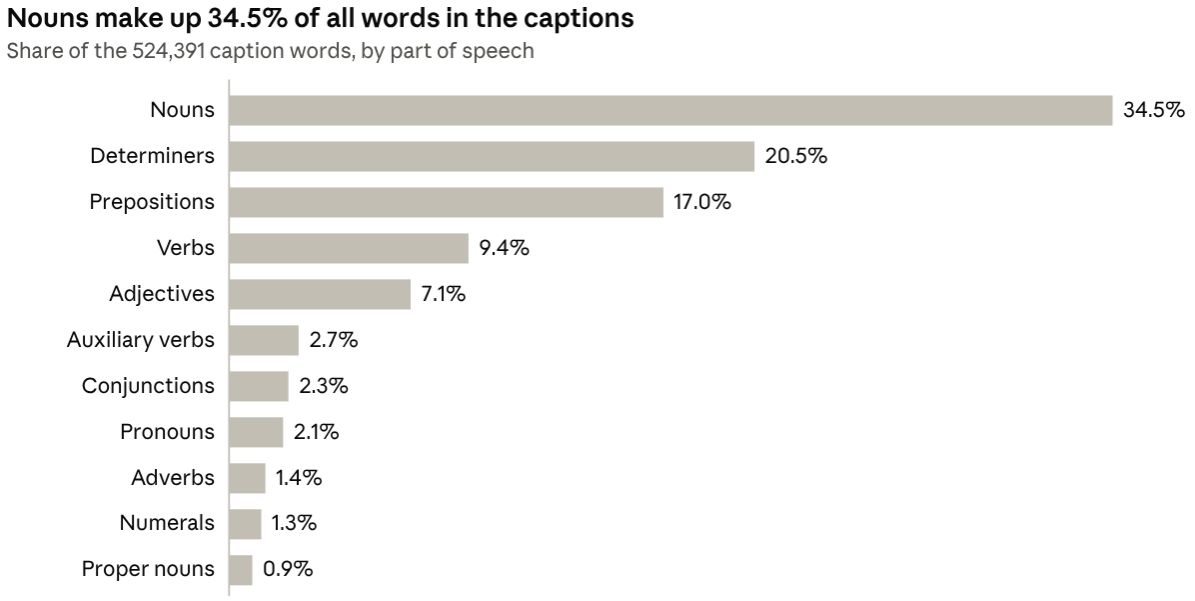


Figure 7. Distribution of parts of speech in the captions.

This profile matches the way people describe pictures: a noun phrase for the main object (a man), a preposition for its place (on a skateboard), a verb for its action (riding) and sometimes an adjective for a colour or size (a white dog). Adverbs, numerals and proper nouns together account for less than four percent.

The most frequent words in each grammatical class, in their base forms, are listed below.

Class	Most frequent words (number of occurrences)
Nouns	man 7,193; people 3,568; woman 3,490; table 2,619; street 2,541
Verbs	sit 5,562; stand 4,251; hold 2,715; rid 1,799; play 1,469
Adjectives	white 2,997; large 2,056; black 1,697; small 1,695; young 1,445

The form rid in the verb row is the base form that the grammar tool produced for riding; it is shown as obtained.

Technical note. Length and vocabulary are computed on the cleaned retrieval captions (50,000 rows) with whitespace tokenisation: total_tokens 524,391, vocabulary 9,770, lexical diversity (vocabulary divided by tokens) 0.0186, mean 10.49, median 10, minimum 6, maximum 46, standard deviation 2.41. Stop words are removed only for the content-word statistics, using the scikit-learn English list. Part-of-speech tagging uses spaCy en_core_web_sm with the parser and named-entity components disabled, applied to the

raw captions in batches of 256, ignoring punctuation and whitespace. Results are in results/eda: caption_statistics.json, word_frequency_full.csv, word_frequency_statistics.json, pos_distribution.csv, pos_statistics.json and the top-20 files for words, nouns, verbs and adjectives.

11. Diversity and relationships

The knowledge base mentions colours and settings often enough to be useful, objects tend to appear in familiar groups, and most pairs of objects in a picture are spread out rather than stacked on each other.

11.1 Colours

The assignment asks for a study of attributes such as colour and shape. G9 counted how many sentences mention each basic colour. A sentence that mentions two colours is counted once for each, so the counts are numbers of sentences, out of 50,000, not numbers of objects. Figure 8 shows the result: white appears in 2,966 sentences, black in 1,675, red in 1,397, blue in 1,128 and green in 1,044, followed by brown (720), yellow (703), orange (423), pink (289), gray (173), grey (135) and purple (134). The two spellings gray and grey together give 308 sentences.

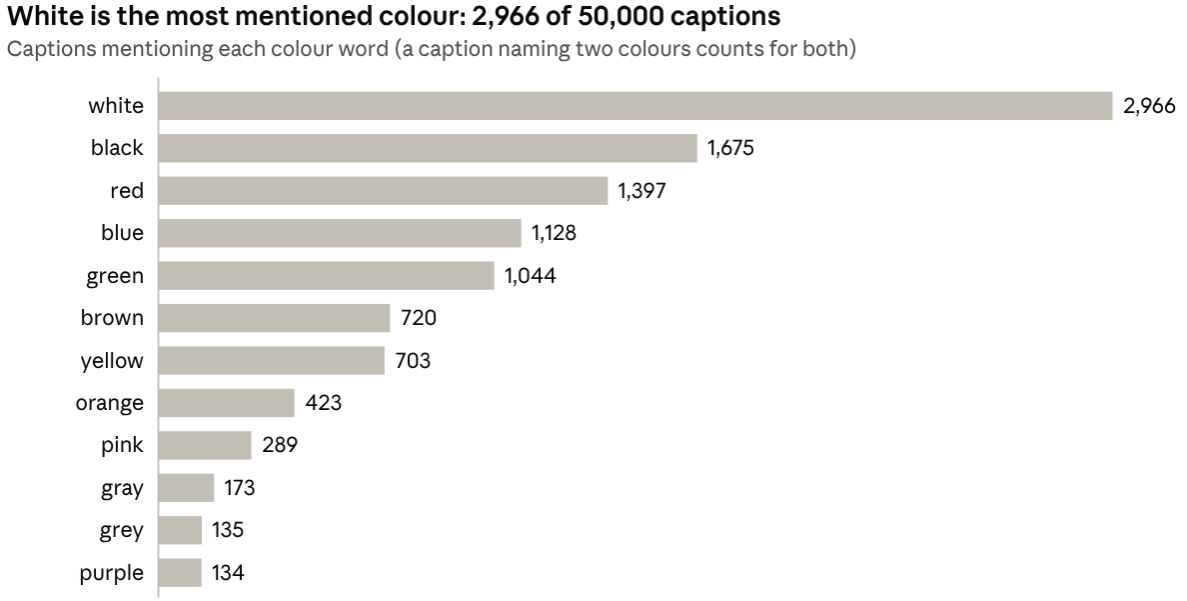


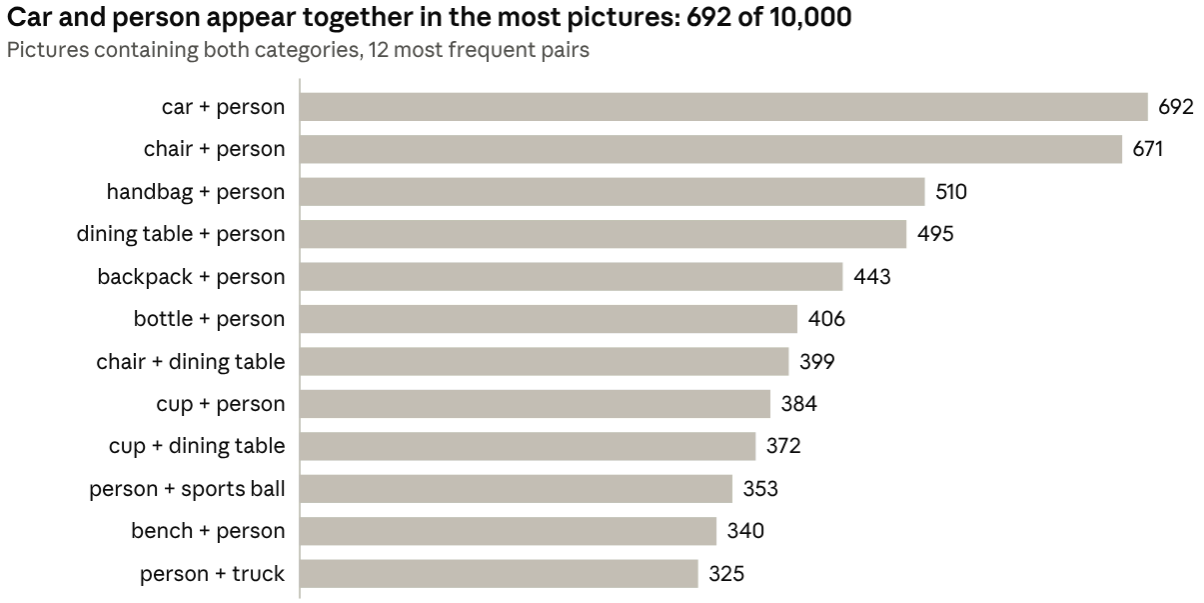
Figure 8. Colour words in the captions.

The ranking is plausible: white and black are the colours of many everyday objects, such as plates, clothes and animals, and also the easiest for annotators to name. A system that retrieves sentences for a white dog will therefore find many candidates, but one that looks for a purple object will find very few.

Shapes, the second attribute named in the assignment, were not analysed separately in this phase; this is listed among the limitations in Chapter 17.

11.2 Objects that appear together

Objects are not scattered at random: tables go with cups, roads with cars. G9 counted, for every pair of categories, the number of pictures in which both appear. Figure 9 shows the twelve most frequent pairs. Almost all involve person, which is expected since person is present in more than half of the pictures: car and person appear together in 692 pictures, chair and person in 671, handbag and person in 510, dining table and person in 495.



top_20_object_pairs.csv, first 12 rows

Figure 9. The twelve most frequent object pairs.

Because person is everywhere, raw counts mostly reflect how common person is. A fairer measure of affinity is the Jaccard similarity: among all the pictures that contain at least one of the two objects, what fraction contain both. On this measure the strongest pairs among the twenty most frequent are cup and dining table (0.275, together in 372 pictures), car and truck (0.245), and chair and dining table (0.245). They correspond to scenes that any person would recognise: a laid table, a road with vehicles.

Overall, 1,864 different pairs of categories are observed. In 7,865 pictures (78.7%) at least two different categories are present, which confirms that the scenes are mostly composed of several kinds of objects.

Technical note. Colour and context cues are computed from the cleaned captions of the retrieval table. A colour is counted when the word is one of the tokens of the sentence; a context cue is counted when one of its phrases occurs in the sentence as a whole phrase. The lists are the colours black, white, red, blue, green, yellow, brown, orange, pink, purple,

gray and grey, and the context phrases given above. Co-occurrence uses the set of distinct categories of each picture; the Jaccard similarity of A and B is the number of pictures containing both divided by the number containing A plus the number containing B minus the number containing both. For the spatial indicators, a pair is horizontally separated when the distance between the box centres on the horizontal axis exceeds a quarter of the sum of the two box widths, vertically separated by the analogous test on heights, and overlapping when the intersection of the two boxes is not empty; crowd annotations and empty boxes are ignored. Results are in colors_caption_cues.csv, contexts_caption_cues.csv, object_cooccurrence_counts.csv, object_cooccurrence_jaccard.csv, object_pairs_full.csv, top_20_object_pairs.csv and spatial_bbox_proxy_counts.csv.

12. Metadata extraction

G9 stored, for every knowledge-base picture, a record of which objects it contains, taken from the human annotations of COCO; richer metadata such as attributes and relationships has not yet been stored per picture.

12.1 What metadata is and why it matters

Metadata is information about the data. A library catalogue card is metadata about a book: author, subject, year. For a picture, metadata can say which objects it contains, what their colours are, how they are arranged and in what setting the scene takes place. The assignment (section 4.4.3, step 5) asks the preprocessing to extract objects, attributes, relationships and context, to store them in JSON or CSV files indexed by picture number, and to document the version and confidence threshold of any detector used.

Metadata is valuable because some of the assigned techniques use it directly. One technique in the assignment, Metadata-Filtered RAG, restricts the search to pictures that have certain visual properties, for example only pictures that contain a dog. Others can use it to expand or check a query.

12.2 What G9 stored

For each of the 10,000 knowledge-base pictures, one row records:

- the picture number, which links the row to the pictures, the sentences and the vectors;
- the number of labelled objects in the picture;
- the number of different categories;
- the list of the categories present, for example a list containing person, chair and dining table.

The information comes from the human annotations of COCO. No object-detection program was run, so there is no detector version and no confidence threshold to document: the labels are not predictions but the work of human annotators, which makes them more reliable than the output of an automatic detector. For 102 pictures the list is empty because COCO has no labelled object there (Chapter 5).

Technical note. The file is metadata/kb_object_metadata.csv with the columns image_id, object_count, distinct_categories and object_categories (a JSON-encoded list of category names, sorted alphabetically). It is built from kb_instances.json after asserting that the identifiers equal the knowledge-base split; all 80 categories occur. Aggregate files are written to results/eda (object_statistics.json and others). The picture number is an integer in every table, so tables can be joined directly with the caption table, the vector identifier file and the manifest.

13. Turning images and words into vectors

G9 translated the 10,000 pictures and 50,000 sentences of the knowledge base into lists of 512 numbers with the CLIP model, so that a computer can measure how similar any two of them are.

13.1 The idea of a shared language

Chapter 2 introduced embeddings. In practice, this stage is a translation. The cleaned pictures of Chapter 7 are given to the image half of CLIP, which returns one list of 512 numbers per picture. The sentences of Chapter 8 are given to the text half of CLIP, which returns one list of 512 numbers per sentence. Because the two halves were trained together, the lists live in the same space: the numbers of a picture of a dog on a street and the numbers of the sentence a dog sleeping on a street are close to each other, while the numbers of a picture of a kitchen are far away.

The result is that three kinds of questions can be answered with the same arithmetic: which sentences resemble this sentence, which pictures resemble this picture, and which sentences describe this picture.

13.2 How the work was organised

Computing 60,000 vectors is a substantial task for a free cloud computer, so it was organised to be safe against interruptions.

- **Pictures** were processed archive by archive, 32 at a time, using the same preprocessing as Chapter 7. The result of each archive was saved before moving to the next, and at the end all parts were merged and sorted by picture number.

- **Sentences** were processed in blocks of 2,000, 128 at a time, from the original text with a maximum length of 77 tokens. Each block was saved on completion and then merged. The 50,000 sentence vectors were produced in 39 seconds on the Tesla T4 graphics processor.
- **Settings** were written to a configuration file: the model name, the vector size, the normalisation, the sentence source and length limit, the batch size and the number of files.

13.3 What was produced

File	Content	Size
kb_image_embeddings.npy	One vector per knowledge-base picture	10,000 by 512
kb_image_ids.npy	The picture number of each row	10,000 values
kb_text_embeddings.npy	One vector per retrieval sentence	50,000 by 512
kb_text_metadata.csv	Picture number, annotation number, original and cleaned text of each row	50,000 rows
embedding_config.json	Model, vector size, settings	one small file

The order of the rows is fixed and documented: the image vectors are sorted by picture number, and the sentence vectors are sorted by picture number and then annotation number, with the same order in the metadata table. This is how each vector can be traced back to its picture and its sentence.

Technical note. The model is openai/clip-vit-base-patch32 loaded with Hugging Face transformers (version 5.20.2), in evaluation mode with inference mode on a Tesla T4. Image features come from get_image_features applied to the output of CLIPImageProcessor; text features come from get_text_features applied to the CLIP tokenizer output; both are read from the pooled, projected output, converted to float32 and normalised with torch.nn.functional.normalize (p = 2). The text-embedding step was run a second time after correcting how the projected vectors are read from the model output; the saved files come from that corrected run. Image shards are stored as part_XXX.npz files with the arrays ids and vectors, text shards as part_NNNNN.npy files, in the folders image_shards and text_shards. The saved embedding_config.json records

the model, transformers version, dimension, counts, normalisation, text source, maximum length, text batch size and data type; the image batch size (32) does not appear in the final file and is recorded here (Chapter 17).

14. Building the knowledge base

The knowledge base consists of three indexes, two exact vector indexes for meaning and one word index, and each of them was built in about one second or less.

14.1 The three indexes

Index	What it contains	Used for	Size	Build time
Picture index (FAISS)	10,000 picture vectors	Finding pictures that look like the query picture	20.5 MB	0.014 s
Sentence index (FAISS)	50,000 sentence vectors	Finding sentences whose meaning fits a picture or a text	102.4 MB	0.153 s
Word index (BM25)	The 50,000 cleaned sentences	Finding sentences that share exact words with a query text	4.7 MB	1.011 s

Both vector indexes are exact: for a query they compare it with every stored vector and return the true nearest ones. The measure is the inner product of normalised vectors, which equals the cosine similarity of Chapter 2. For a collection of this size an exact search is fast and has the advantage of making no approximation errors; approximate indexes become necessary only with millions of vectors. The word index is the BM25 index of Chapter 2.

14.2 How a query travels through the library

Figure 10 shows how the three indexes work together in G9's hybrid retrieval test, which is described in the next chapter. A query picture is turned into a vector by CLIP and used in two searches: one among the picture vectors and one among the sentence vectors. In parallel, a short preliminary sentence describing the query picture is produced by a captioning model and used as a text query in the word index. The three rankings are then merged by reciprocal rank fusion, and the best pictures, with their sentences, are returned as evidence.

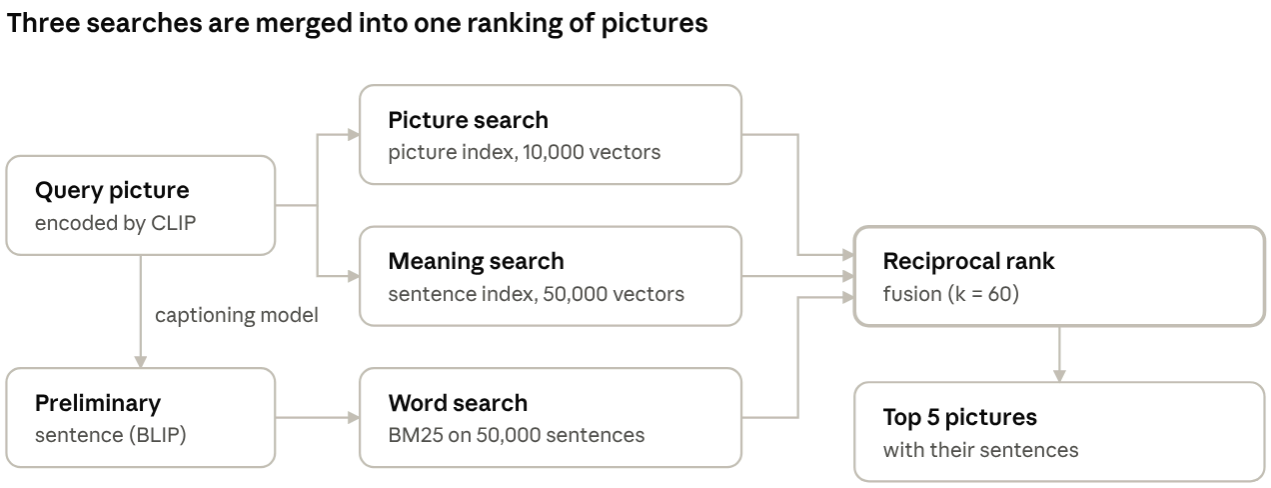


Figure 10. Hybrid retrieval: three searches merged by reciprocal rank fusion.

Technical note. The vector indexes are faiss.IndexFlatIP with dimension 512, built from float32 arrays that are normalised again with faiss.normalize_L2 before insertion, and saved as indexes/faiss_images.index (20,480,045 bytes) and indexes/faiss_captions.index (102,400,045 bytes). The word index is rank_bm25.BM25Okapi built on the whitespace tokens of caption_clean and saved with pickle as indexes/bm25.pkl (4,734,813 bytes); because pickle files can run code when loaded, they should be loaded only from trusted sources, as the notebook notes. Statistics are saved in results/retrieval/index_statistics.json. In the retrieval test, 100 candidates were taken from the picture index, 300 sentence results were merged into unique pictures from the sentence index, and up to 300 from BM25, before fusion.

15. Testing the whole chain

Two small retrieval tests, one starting from a sentence and one from a picture, show that the files, vectors and indexes fit together and return sensible evidence.

15.1 Test A: starting from a sentence

The query was the sentence a person riding a bicycle on a street. It was searched by meaning in the sentence index and by words in the word index, and the two rankings were merged by reciprocal rank fusion. The five pictures returned, with the sentence that matched, were:

Rank	Picture	Matching sentence	Fusion score
1	117325	A person on a bicycle is riding along a road.	0.0313

Rank	Picture	Matching sentence	Fusion score
2	281227	A person on a bicycle in a bicycle lane.	0.0308
3	536827	a person riding a bike on a city street	0.0305
4	162355	A man with a bag riding a bicycle in the street.	0.0294
5	447376	A person riding a motorcycle on a street.	0.0293

Four of the five results describe exactly the requested scene. The fifth, a motorcycle, is a near miss: it shares most words and its meaning is close. This is typical of retrieval and a reminder that the evidence handed to the caption generator is helpful but not perfect.

15.2 Test B: starting from a picture

The second test is closer to the real use. A picture from the test set was taken as the query. First, a captioning model (BLIP) wrote a preliminary sentence for it: a group of people walking up a snow covered mountain. Then three searches were run: pictures similar to the query picture, sentences whose meaning fits the query picture, and sentences sharing words with the preliminary sentence. Their rankings were merged by reciprocal rank fusion. The five best results were:

Rank	Picture	Matching sentence	Fusion score	Picture similarity
1	457796	A group of men walking across a snow covered slope.	0.0450	0.827
2	81961	A group of people on skis on a snow covered slope.	0.0402	0.857
3	448117	A group of people carrying ski equipment while walking on snow covered ground.	0.0389	0.811
4	390856	A group of people on skis standing at the peak of a mountain.	0.0386	0.841
5	290332	A group of people on a snow covered ground skiing.	0.0382	0.795

All five matching sentences describe groups of people in snow, consistent with the preliminary sentence. Note that the first place is not the picture with the highest visual similarity (that is the second place, 0.857): the fusion rewards the picture that is good on several rankings at once, which is the purpose of a hybrid design.

Technical note. The preliminary sentence is generated by Salesforce/blip-image-captioning-base with beam search (3 beams, up to 35 new tokens). Reciprocal rank fusion uses k = 60. Test A fuses the top 100 results of the dense sentence search and the top 100 BM25 results and keeps five distinct pictures. Test B fuses three rankings: the 100 nearest picture vectors, the first 300 sentence results from the sentence index reduced to unique pictures, and up to 300 BM25 results reduced to unique pictures, with the query tokens extracted from the preliminary sentence. Outputs are stored in results/retrieval (hybrid_retrieval_demo.csv, image_query_hybrid_results.csv, their configuration files and the figure image_query_hybrid_visualization.png). The configuration file of Test B records uses_test_reference_captions as false.

16. Quality control and audit results

The final audit passes all 13 automated checks, and shows that the test set resembles COCO′s validation part closely, though it is a random draw and not a stratified one.

16.1 What the final audit does

The last notebook does not download or retrain anything. It inspects the files that the earlier stages produced and answers one question: are they consistent with each other and with the rules of the assignment? It also adds the remaining statistics, and writes a release record, a short readme and a fingerprint (SHA-256 hash) of the three small control files, so that any later change to them can be detected. Its overall verdict is deliberately cautious: ready for review, not frozen.

16.2 The thirteen automated checks

#	Check	What it verifies	Result
1	Split counts	The three sets have 10,000, 100 and 100 pictures	Pass
2	Split disjoint	No picture belongs to two sets	Pass
3	Manifest	The manifest has 10,200 rows consistent with the split file	Pass
4	Five sentences each	50,000 sentences, exactly five for each knowledge-base picture, and only for those pictures	Pass
5	Annotations scoped	Object annotations exist only for knowledge-base pictures	Pass
6	Image preprocessing	All 10,200 pictures passed the image audit	Pass

#	Check	What it verifies	Result
7	Picture vectors	10,000 vectors of 512 numbers, matching the knowledge-base pictures	Pass
8	Sentence vectors	50,000 vectors of 512 numbers and the metadata file present	Pass
9	Vector quality	No missing or infinite value; every vector has length one	Pass
10	Vector indexes	The two indexes hold 10,000 and 50,000 vectors of 512 numbers	Pass
11	Word index	The BM25 file exists	Pass
12	Archives	The archives contain exactly the 10,200 files of the manifest	Pass
13	Duplicates	No byte-identical picture appears in two sets	Pass

The record states all_automated_checks_pass as true, the status as ready for review and not frozen, and lists three pending items: the confirmation of the official shared test set, the confirmation of context and attribute labels if the teachers require visually verified ones, and the detection of near-duplicate pictures.

16.3 Is the test set representative?

The assignment asks for a stratified test set. G9 drew a random one, and the audit measures how representative it turned out to be. For each of the 80 categories it compares the share of pictures that contain the category in the whole COCO validation part (5,000 pictures) with the share in the 100 test pictures. The largest differences are:

Category	Share in COCO validation part	Share in test set	Difference
truck	5.0%	8.0%	3.0 points
pizza	3.1%	6.0%	2.9 points
fork	3.1%	6.0%	2.9 points
person	53.9%	51.0%	2.9 points
sink	3.7%	1.0%	2.7 points

No difference exceeds 3.0 percentage points, so the category mix of the test set is close to that of the validation part. However, eight categories do not appear in the test set at all: baseball glove, airplane, baseball bat, bear, toothbrush, scissors, hair drier and toaster. This is what chance produces with small samples. A category present in 2% of pictures is missing from a random sample of 100 about one time in eight, so some absences among 80 categories are to be expected. A stratified draw would avoid them by design.

The finding supports two statements: the random test set is a fair but imperfect sample, and a stratified redraw, if the teachers require it, is feasible without touching the knowledge base.

Technical note. The audit notebook reads results/qa/image_preprocessing_audit.csv, the split file, the manifest, the retrieval caption table, kb_instances.json, the embeddings (read in blocks of 2,048 rows; length tolerance 0.002), the two FAISS indexes and the archives. Duplicate screening groups ZIP entries by CRC-32 and size and confirms with SHA-256. The stratification diagnostic is saved as results/qa/test_stratification_diagnostic.csv with the columns category, val2017_images, test_images, val_prevalence, test_prevalence and absolute_prevalence_gap. Release files are results/preprocessing_release/preprocessing_audit.json, README_preprocessing.md and small_file_sha256.json. The caption cue statistics and the box-geometry proxies are also produced in this notebook (Chapter 11).

17. Limitations and open issues

Two open decisions prevent G9 from declaring the release frozen, and several smaller gaps are known and documented here rather than hidden.

17.1 Decisions that depend on the teachers

The size and nature of the test set. The assignment says 100 test pictures in sections 4.1 and 4.2 and 1,000 in sections 4.4.2 and 4.4.3 (one line reads 1,00), and it also asks for a stratified subset of the COCO validation part. G9′s current test set is 100 random pictures. If a different set is required, only the split files, the manifest, the test archive and the check that validation and test do not overlap must be regenerated. The knowledge base and its vectors and indexes are not affected, because they come from the other COCO part. The question should be put to the teachers in writing.

The format and content expected on 4 November. The assignment documents the exploration and preprocessing but does not state in what form the phase is to be handed in. This report and the shared repository are G9′s proposal.

17.2 Known limits to keep in mind

- **Cropping.** The CLIP preprocessing keeps the central square of each picture, so very wide or tall pictures lose their sides.
- **Apostrophes.** Words such as don't are split into two tokens in the word index.
- **Near-duplicates.** Only byte-identical pictures were searched for; the same scene saved twice with small differences would not be detected.
- **Pretrained models.** The models used may have seen COCO during their own training, which could favour results in a way that no preprocessing can prevent.
- **Size.** Ten thousand pictures is a small library. Rare categories have few neighbours, which will limit the quality of captions for unusual scenes.

Technical note. The audit file preprocessing_audit.json lists the same pending items under known_pending, and its status field is READY_FOR_REVIEW_NOT_FROZEN. Changing the test set requires regenerating split_ids.json, image_manifest.csv, the test archive and the audit files; the audit notebook states that identifiers must never be changed without regenerating the artifacts that depend on them.

18. Conclusion and next steps

G9 has built, checked and documented a complete data foundation for hybrid retrieval-augmented captioning, and what remains before the freeze is a short list of decisions and verifications.

18.1 What was achieved

Starting from the raw COCO 2017 collection, the group selected three separate sets of pictures with a repeatable random draw, collected and audited their files, prepared the pictures and the sentences according to the rules of the assignment, explored the data statistically, converted the pictures and sentences to vectors with CLIP, and built the indexes of a knowledge base. A final audit confirms that all 13 automated checks pass, that no picture is shared between sets, and that no sentence of the evaluation pictures is in the library.

Three features of the work deserve to be highlighted. The first is the separation of originals and cleaned copies, which keeps the evaluation honest. The second is the layered protection against leakage, which starts from the structure of the data itself and is confirmed by automatic checks. The third is the habit of recording: settings, counts and verdicts are saved in files beside the results, so that any claim in this report can be traced to a file.

18.2 Next steps

1. **Settle the test set.** Ask the teachers whether the test set must have 100 or 1,000 pictures and whether it must be stratified; regenerate the test files if needed.
2. **Use the validation set.** Run the small comparison of preprocessing options on the 100 validation pictures and record the decision.
3. **Close the documentation gaps.** Complete the embedding configuration, add the scatter plot and the shape analysis, run the vector alignment check, and decide how to supply per-picture metadata.
4. **Freeze and share.** Once the points above are settled, mark the audit as frozen, store the fingerprints of the control files and share the release with the other groups.
5. **Move to the next phase.** The assignment schedules model training and validation after the professors validate the preprocessing; the frozen knowledge base is the input of that phase.

In short, the foundation is solid and honest about its gaps. What it needs now is the teachers' decision on the test set and a small validation experiment, after which it can be frozen and handed to the other groups.

Appendix A: Technical reference

This appendix collects, for readers who want the exact details, the settings and key numbers of the preprocessing release. Paths are relative to the shared folder G9_Hybrid_RAG.

A.1 Settings

Parameter	Value
Random seed	42
Set sizes	10,000 knowledge base, 100 validation, 100 test
Sources	train2017 (knowledge base), val2017 (validation and test)
Excluded picture	139 (learning example)
Archive batch size	250 pictures, uncompressed
Image model	openai/clip-vit-base-patch32
Image input	224 by 224, CLIP mean and standard deviation normalisation

Parameter	Value
Orientation and colour	EXIF transpose, conversion to RGB
Embedding size and type	512, float32, L2-normalised
Image batch size	32
Text source, batch size, maximum length	caption_raw, 128, 77 tokens
Sentences per picture	5 (lowest annotation numbers)
Sentence cleaning	lowercase, punctuation to space, whitespace collapsed
Vector index	FAISS IndexFlatIP (exact, inner product)
Word index	BM25Okapi on cleaned sentences
Fusion	Reciprocal rank fusion, k = 60
Preliminary caption model	Salesforce/blip-image-captioning-base, 3 beams, 35 new tokens
Part-of-speech model	spaCy en_core_web_sm
Duplicate screening	CRC-32 and size, confirmed by SHA-256
Hardware	Google Colab, Tesla T4 graphics processor